GUÍA DE LABORATORIO PRÁCTICO: POSTGRESQL NATIVO

Programa: Análisis y Desarrollo de Software (ADSO) | Ficha: 3172526 | Modalidad: Taller Práctico de Aula

Escenario de Proyecto: Sistema de Gestión de Biblioteca (BiblioSENA) - DDL, DML, JOINs, PL/pgSQL y Triggers

1. Presentación del Caso de Negocio y Modelo de Datos

Objetivo General del Taller: Capacitar al aprendiz en el diseño, implementación, consulta y automatización de bases de datos relacionales utilizando **PostgreSQL nativo** en el entorno **pgAdmin 4**. Se desarrollan competencias en Lenguaje de Definición de Datos (DDL), Lenguaje de Manipulación de Datos (DML), uniones complejas (JOINs), agrupaciones con filtros avanzados (GROUP BY / HAVING) y programación orientada a la base de datos mediante PL/pgSQL (Procedimientos Almacenados y Triggers).

Escenario de Negocio (BiblioSENA): El Centro de Formación requiere un sistema centralizado para gestionar el inventario de literatura técnica, autores, categorías temáticas y préstamos a aprendices. Para garantizar la consistencia, eliminar la redundancia y cumplir con la **Tercera Forma Normal (3NF)**, el modelo se descompone en 6 tablas interrelacionadas:

Entidad / Tabla	Rol en la Arquitectura	Llaves y Cardinalidad
editoriales	Almacena las casas editoriales de los libros.	PK: id_editorial | Relación 1:N con libros
generos	Clasificación temática (Ingeniería, Novela, etc.).	PK: id_genero | Relación 1:N con libros
autores	Información de los creadores literarios.	PK: id_autor | Relación N:M con libros
libros	Catálogo central de obras con precio y stock.	PK: id_libro, FK: id_editorial, FK: id_genero
libros_autores	Tabla intermedia para resolver la coautoría (N:M).	PK compuesta / FKs: id_libro, id_autor
prestamos	Registro transaccional de circulaciones a usuarios.	PK: id_prestamo, FK: id_libro | Histórico

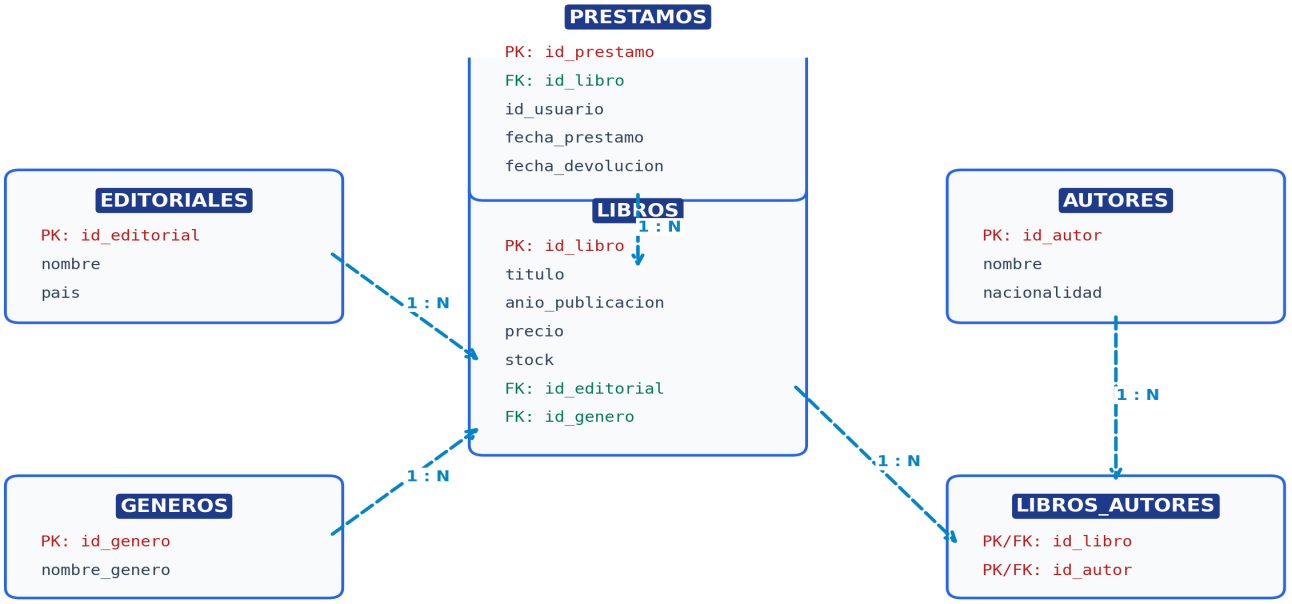


Figura 1: Diagrama Entidad-Relación (ERD) del Sistema BiblioSENA en 3NF.

2. DDL - Creación de la Estructura y Restricciones (CREATE TABLE)

El Lenguaje de Definición de Datos (DDL) define las estructuras donde reside la información. En PostgreSQL, el uso adecuado de tipos de datos nativos y restricciones garantiza la **integridad referencial y de dominio**. A continuación se explican los conceptos aplicados:

Fundamentación Teórica de DDL en PostgreSQL:

- **SERIAL:** Pseudotipo entero autoincrementable que crea implícitamente una secuencia (SEQUENCE).
- **VARCHAR(n):** Cadena de caracteres de longitud variable optimizada con límite de almacenamiento.
- **PRIMARY KEY:** Identificador único no nulo de la tupla.
- **FOREIGN KEY:** Regla de integridad que vincula campos con la PK de la tabla padre, evitando huérfanos.
- **CHECK:** Restricción lógica de validación (Ej: *stock >= 0*, *precio > 0*).

```
-- ============================================================================
-- SCRIPT DDL: CREACIÓN DE ESTRUCTURAS - SISTEMA BIBLIOSENA
-- ============================================================================

-- 1. Tabla Editoriales (Padre)
CREATE TABLE editoriales (
    id_editorial SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    pais VARCHAR(50) DEFAULT 'Colombia'
);

-- 2. Tabla Géneros (Padre)
CREATE TABLE generos (
    id_genero SERIAL PRIMARY KEY,
    nombre_genero VARCHAR(50) NOT NULL UNIQUE
);

-- 3. Tabla Autores (Padre)
CREATE TABLE autores (
    id_autor SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    nacionalidad VARCHAR(50)
);

-- 4. Tabla Libros (Hija de Editoriales y Géneros)
CREATE TABLE libros (
    id_libro SERIAL PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    anio_publicacion INT CHECK (anio_publicacion > 1500),
    precio NUMERIC(10,2) CHECK (precio >= 0),
    stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0), -- Campo Stock para control
    id_editorial INT NOT NULL,
    id_genero INT NOT NULL,
    CONSTRAINT fk_libros_editoriales FOREIGN KEY (id_editorial)
        REFERENCES editoriales(id_editorial) ON DELETE RESTRICT,
    CONSTRAINT fk_libros_generos FOREIGN KEY (id_genero)
        REFERENCES generos(id_genero) ON DELETE RESTRICT
);

-- 5. Tabla Intermedia Libros_Autores (Resolución N a M)
CREATE TABLE libros_autores (
    id_libro INT NOT NULL,
    id_autor INT NOT NULL,
    PRIMARY KEY (id_libro, id_autor), -- Llave primaria compuesta
    CONSTRAINT fk_la_libro FOREIGN KEY (id_libro)
        REFERENCES libros(id_libro) ON DELETE CASCADE,
    CONSTRAINT fk_la_autor FOREIGN KEY (id_autor)
        REFERENCES autores(id_autor) ON DELETE CASCADE
);

-- 6. Tabla Préstamos (Transaccional)
CREATE TABLE prestamos (
    id_prestamo SERIAL PRIMARY KEY,
    id_libro INT NOT NULL,
    id_usuario INT NOT NULL, -- ID del Aprendiz o Lector
    fecha_prestamo DATE DEFAULT CURRENT_DATE,
    fecha_devolucion DATE,
    CONSTRAINT fk_prestamos_libro FOREIGN KEY (id_libro)
        REFERENCES libros(id_libro) ON DELETE RESTRICT
);
```

3. DML - Inserción de Datos Coherentes (INSERT INTO)

Para poblar la base de datos sin violar las restricciones de llave foránea, el orden de inserción debe seguir la jerarquía de dependencias: **1° Tablas Padre (Sin FK) $\rightarrow$ 2° Tablas Hijas $\rightarrow$ 3° Tablas Intermedias/Transaccionales**.

```
-- ============================================================================
-- SCRIPT DML: POBLAMIENTO INICIAL DE DATOS
-- ============================================================================

-- Inserción en Tablas Padre
INSERT INTO editoriales (nombre, pais) VALUES
('Alfaomega', 'México'),
('Prentice Hall', 'Estados Unidos'),
('McGraw-Hill', 'España');

INSERT INTO generos (nombre_genero) VALUES
('Programación y Software'),
('Bases de Datos'),
('Inteligencia Artificial');

INSERT INTO autores (nombre, nacionalidad) VALUES
('Blas León Romero', 'Española'),
('Abraham Silberschatz', 'Estadounidense'),
('Andrew S. Tanenbaum', 'Holandesa');

-- Inserción en Catálogo de Libros (Relacionando Editoriales y Géneros)
INSERT INTO libros (titulo, anio_publicacion, precio, stock, id_editorial, id_genero) VALUES
('El libro práctico de Bases de datos', 2020, 85000.00, 5, 1, 2),
('Fundamentos de Sistemas de Bases de Datos', 2018, 120000.00, 3, 2, 2),
('Sistemas Operativos Modernos', 2015, 95000.00, 4, 3, 1);

-- Inserción en Coautoría (Relación N:M)
INSERT INTO libros_autores (id_libro, id_autor) VALUES
(1, 1), -- Blas León escribió El libro práctico...
(2, 2), -- Silberschatz escribió Fundamentos...
(3, 3); -- Tanenbaum escribió Sistemas Operativos...

-- Inserción en Préstamos (Transacciones iniciales)
INSERT INTO prestamos (id_libro, id_usuario, fecha_prestamo) VALUES
(1, 3172526, CURRENT_DATE - INTERVAL '5 days'),
(2, 3172526, CURRENT_DATE - INTERVAL '2 days'),
(3, 1098765, CURRENT_DATE);
```

4. DML - Operaciones de Actualización y Eliminación Segura

Las sentencias UPDATE y DELETE modifican el estado persistente. En entornos de producción, **siempre debe utilizarse la cláusula WHERE** para acotar el impacto a registros específicos.

```
-- 1. Ejercicio Resuelto UPDATE: Modificar el nombre y país de la Editorial 'Alfaomega'
UPDATE editoriales
SET nombre = 'Alfaomega Colombiana S.A.',
    pais = 'Colombia'
WHERE id_editorial = 1;

-- 2. Ejercicio Resuelto DELETE: Eliminar un préstamo registrado de forma errónea
DELETE FROM prestamos
WHERE id_prestamo = 3;
```

5. El Poder de las Uniones de Tablas (SQL JOINs)

Las consultas relacionales permiten reconstruir la información dispersa en múltiples tablas normalizadas mediante operaciones algebraicas de combinación. A continuación se presentan las variantes de combinaciones:

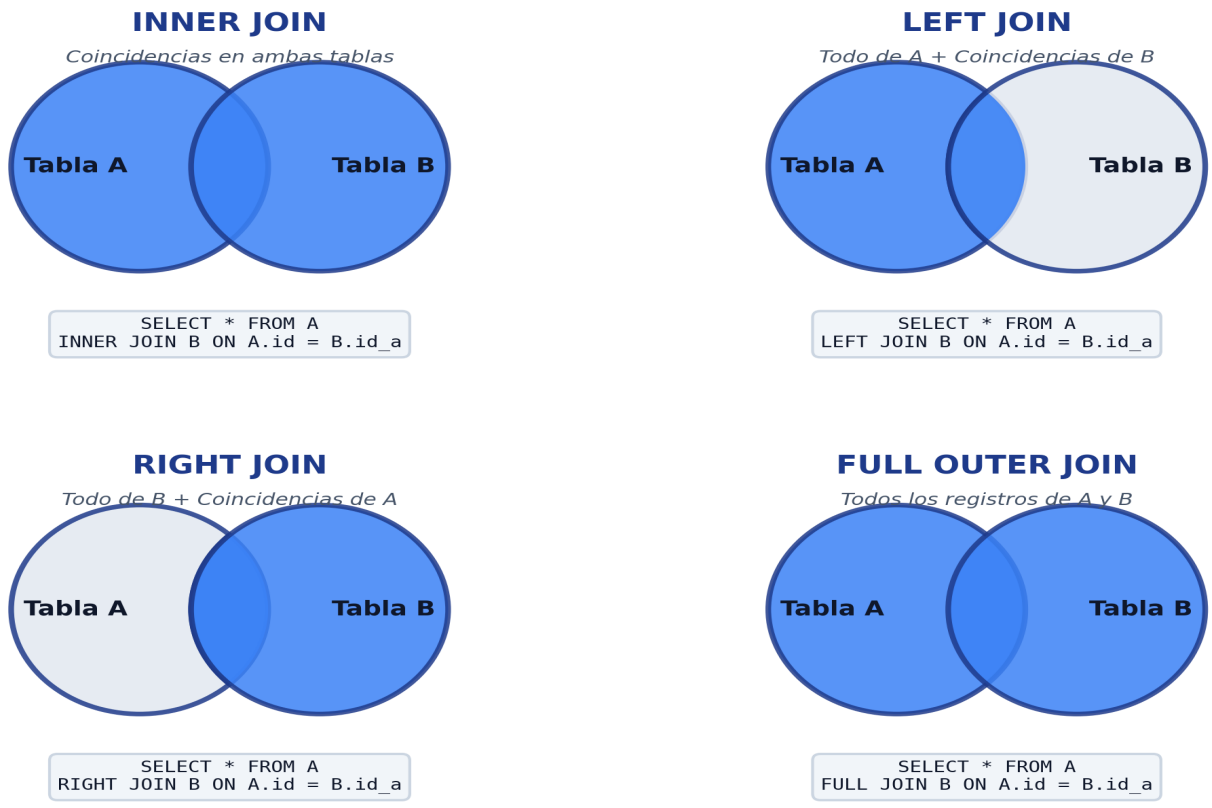


Figura 2: Representación gráfica de conjuntos para las operaciones JOIN en SQL.

Consulta Maestra (Requerimiento Principal INNER JOIN):

Obtener un listado consolidado que muestre el Título del libro, el Nombre de la Editorial, el Nombre del Género y el Nombre de los Autores asociados mediante el cruce de las 5 tablas relacionales:

```
-- CONSULTA MAESTRA INNER JOIN (5 TABLAS UNIFICADAS)
SELECT
    l.id_libro,
    l.titulo AS "Título del Libro",
    e.nombre AS "Editorial",
    g.nombre_genero AS "Género",
    a.nombre AS "Autor",
    l.stock AS "Stock Disponible"
FROM libros l
INNER JOIN editoriales e ON l.id_editorial = e.id_editorial
INNER JOIN generos g ON l.id_genero = g.id_genero
INNER JOIN libros_autores la ON l.id_libro = la.id_libro
INNER JOIN autores a ON la.id_autor = a.id_autor
ORDER BY l.titulo ASC;
```

Diferencias entre Variaciones de JOINs con Ejemplos SQL:

```
-- LEFT JOIN: Muestra TODOS los libros, tengan o no préstamos registrados
SELECT l.titulo, p.id_prestamo, p.fecha_prestamo
FROM libros l
LEFT JOIN prestamos p ON l.id_libro = p.id_libro;

-- RIGHT JOIN: Muestra TODAS las editoriales, incluso las que aún no tienen libros en catálogo
SELECT e.nombre AS editorial, l.titulo
FROM libros l
RIGHT JOIN editoriales e ON l.id_editorial = e.id_editorial;

-- FULL OUTER JOIN: Auditoría total - Muestra todos los libros y préstamos coincidan o no
SELECT l.titulo, p.id_prestamo, p.id_usuario
FROM libros l
FULL OUTER JOIN prestamos p ON l.id_libro = p.id_libro;
```

6. Agrupaciones y Filtros Transaccionales (GROUP BY y HAVING)

El orden lógico de ejecución en SQL procesa las cláusulas en la siguiente secuencia: **FROM $\rightarrow$ WHERE $\rightarrow$ GROUP BY $\rightarrow$ HAVING $\rightarrow$ SELECT $\rightarrow$ ORDER BY**. Mientras que WHERE filtra filas individuales antes de agrupar, HAVING evalúa condiciones sobre los grupos consolidados.

```
-- Consulta: Cantidad de libros y valor total por género (Solo géneros con más de 1 libro)
SELECT
    g.nombre_genero AS "Género",
    COUNT(l.id_libro) AS "Total Libros",
    SUM(l.stock) AS "Stock Total",
    ROUND(AVG(l.precio), 2) AS "Precio Promedio"
FROM generos g
INNER JOIN libros l ON g.id_genero = l.id_genero
GROUP BY g.id_genero, g.nombre_genero
HAVING COUNT(l.id_libro) > 1
ORDER BY "Total Libros" DESC;
```

7. Programación en Servidor: Procedimientos Almacenados (PL/pgSQL)

Los procedimientos almacenados encapsulan la lógica de negocio directamente en el motor de base de datos, reduciendo el tráfico de red y garantizando ejecuciones atómicas bajo el cumplimiento de los principios ACID.

```
-- PROCEDIMIENTO ALMACENADO: REGISTRO DE PRÉSTAMOS
CREATE OR REPLACE PROCEDURE sp_registrar_prestamo(
    p_id_libro INT,
    p_id_usuario INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_stock INT;
BEGIN
    -- 1. Validar existencia y consultar stock del libro
    SELECT stock INTO v_stock
    FROM libros
    WHERE id_libro = p_id_libro;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El libro con ID % no existe en el catálogo.', p_id_libro;
    END IF;

    -- 2. Validar disponibilidad de inventario
    IF v_stock <= 0 THEN
        RAISE EXCEPTION 'No hay ejemplares disponibles en stock para el libro ID %.', p_id_libro;
    END IF;

    -- 3. Registrar la transacción en la tabla prestamos
    INSERT INTO prestamos (id_libro, id_usuario, fecha_prestamo)
    VALUES (p_id_libro, p_id_usuario, CURRENT_DATE);

    RAISE NOTICE 'Préstamo registrado exitosamente para el usuario %.', p_id_usuario;
END;
$$;

-- Ejemplo de Ejecución en pgAdmin:
-- CALL sp_registrar_prestamo(1, 3172526);
```

8. Automatización de Eventos mediante Disparadores (Triggers)

Un **Trigger** es un objeto de base de datos que ejecuta automáticamente una función PL/pgSQL cuando ocurre un evento DML (INSERT, UPDATE, DELETE). En PostgreSQL se compone de dos elementos independientes: la **Función Ejecutora** (que retorna el tipo TRIGGER) y la **Declaración del Disparador**.



Figura 3: Flujo de ejecución sincrónico entre Procedimiento, Inserción y Trigger.

```
-- 1. Función del Trigger que descuenta stock
CREATE OR REPLACE FUNCTION fn_actualizar_stock_prestamo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Descontar 1 unidad en el stock del libro prestado
    UPDATE libros
    SET stock = stock - 1
    WHERE id_libro = NEW.id_libro;

    RETURN NEW;
END;
$$;

-- 2. Definición del Disparador vinculado a la tabla prestamos
CREATE OR REPLACE TRIGGER trg_actualizar_stock_prestamo
AFTER INSERT ON prestamos
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_stock_prestamo();
```

9. Retos Prácticos de Aplicación en Aula (Para el Aprendiz)

A continuación, el aprendiz debe ejecutar los siguientes ejercicios en pgAdmin 4 para afianzar los conceptos estudiados:

- **Reto 1 (DDL & Alter):** Agregar un campo de tipo *VARCHAR(20)* llamado *estado* a la tabla *prestamos* con un valor por defecto 'Activo'.
- **Reto 2 (Consultas JOIN):** Escribir una consulta con *LEFT JOIN* que muestre los aprendices/usuarios y los libros que tienen en préstamo actualmente.
- **Reto 3 (Agrupaciones):** Consultar cuál es la editorial que tiene el mayor valor total acumulado en inventario (SUM(precio * stock)).
- **Reto 4 (PL/pgSQL):** Crear un procedimiento almacenado *sp_devolver_prestamo(p_id_prestamo INT)* que registre la fecha de devolución y sume +1 al stock del libro.
- **Reto 5 (Control de Excepciones):** Modificar el Trigger para que verifique si el stock llega a cero y genere una alerta por consola (RAISE NOTICE).